Web Application Penetration Testing & Vulnerability Assessment Project report.

B00174326

Sorin Bivol

Vulnerability one:

A) Vulnerability Description and root cause

The "Login Bjoern" challenge requires logging into Bjoern's account without changing his password, using SQL injection, or hacking his Google account. The solution is not a brute force or credential stuffing attack – the password token is actually exposed in the application's source code. This is a hardcoded credentials issue (CWE-798) falling under OWASP A07:2025 (Authentication Failures). The root cause is that a password reset token or authentication credential was left in a client-side file where anyone inspecting the source could find it. Developers sometimes embed tokens in frontend code for testing or convenience, forgetting that these files are downloaded to every user's browser. In this case, Bjoern's password equivalent was sitting in plain text within a JavaScript file or HTML source.

B) Investigation Process

I started by opening Chrome DevTools (F12) and navigating to the Sources tab. I looked through all the JavaScript files loaded by the application, paying attention to any files with names suggesting configuration, user data, or reset functionality. I also viewed the page source (Ctrl+U) to check for any HTML comments or embedded scripts. After some searching, I found a file that contained what looked like a password reset token or an authentication secret associated with Bjoern. The token was in plain text, not hashed or obfuscated. I copied the token and tried using it on the login page. Depending on the challenge's exact mechanism, I either pasted the token directly into the password field or used it in a specific request format. It worked.

C) How the exploit works

The exploit is straightforward once you know where to look. The application serves JavaScript files to the client that contain hardcoded authentication data. In this case, Bjoern's password token was present in one of these files. To execute the attack:

1. Open the application in Chrome or Firefox
2. Open Developer Tools (F12) and go to the Sources tab
3. Browse through the loaded JavaScript files

4. Locate the file containing the token
5. Copy the token string
6. Return to the login page and enter Bjoern's username
7. Paste the token into the password field or use it in the appropriate request

No password cracking, no injection, no brute force – just reading what the developers accidentally left exposed. Screenshots attached showing the token in the source and the successful login.

D) Potential impact

Hardcoded authentication tokens in client-side code are a serious security flaw. In a real-world application, this could expose password reset tokens, API keys, session secrets, or OAuth credentials. An attacker who finds such a token could impersonate the user, access their data, or perform actions on their behalf. If the token belongs to an administrator, the impact is catastrophic – full system compromise. Real-world examples include numerous mobile applications where API keys were found embedded in the app binary, allowing attackers to abuse backend services at the developer's expense. Even if the exposed token has limited privileges, it can be a starting point for privilege escalation or lateral movement within the application.

E) Recommended Mitigation

Authentication secrets, password reset tokens, and credentials should never be stored in client-side code. These secrets must be generated on the server and delivered to the client only through secure, authenticated channels (e.g., as HTTP-only cookies or session-based tokens). If a password reset token needs to be used by the client, it should be delivered via email or SMS, not embedded in JavaScript files. For development and testing, use environment variables or configuration files that are excluded from version control and not deployed to production. Automated secret scanning tools in CI/CD pipelines can detect accidental commits of secrets before they reach production. This aligns with OWASP ASVS requirement 2.2.1 and the secure design principle of minimizing attack surface by not exposing secrets to the client.

Vulnerability two:

A) Vulnerability Description and root cause

The "Reset Jim's Password" challenge exploits a weak security question in the password reset mechanism. When a user requests a password reset, the application asks a security question instead of sending a reset link to a verified email address. The problem is that the answer to Jim's

security question is publicly discoverable within the application. This is classified as CWE-640 (Weak Password Recovery Mechanism for Forgotten Password) and falls under OWASP A07:2025 (Authentication Failures). The root cause is the use of knowledge-based authentication (security questions) for account recovery. Security questions often have answers that can be guessed, found on social media, or discovered through OSINT. In this case, the answer was directly visible in content Jim had posted elsewhere in the application.

B) Investigation Process

I began by clicking the "Forgot Password" link on the login page and entering Jim's email address. I needed to find his email address first – I found it in the adminstration page when i was logged in as admin. The system presented a security question. I had no prior knowledge of the answer, so I started searching for an answer in the application for any information related to Jim. I checked his user profile page, any product reviews or comments he had written, the "About Us" page, and any forum-style sections where users could leave public content. After about ten minutes of searching, I found a comment from Jim that contained the word “Starfleet” So i googled him and found him on Wikipedia. I could see the answer in front of me. Samuel. That was his brother's name.

C) How the exploit works

The attack follows the standard password reset flow but with reconnaissance added. First, locate the target user's email address or username from publicly available information within the application (e.g., reviews, comments, profile pages). Second, initiate the password reset process and provide the user's email address. Third, when the security question appears, search for the application for any content the user has generated that might contain the answer. Fourth, enter the discovered answer. Fifth, set a new password. Sixth, log in as the user. The entire attack can be performed using only a web browser. No automated tools are required, though tools like Burp Suite could speed up the reconnaissance phase.

D) Potential impact

Weak password reset mechanisms are a common real-world vulnerability. Attackers can take over accounts by answering easy security questions, often using information found on social media. The 2016 Snapchat breach involved attackers using a separate service leaked database to answer security questions and reset user passwords. Celebrity account takeovers on Twitter and Instagram have frequently occurred because security question answers (e.g., mother's maiden name, pet's name, birthplace) were publicly available. The impact is account takeover, which can lead to data theft, fraudulent transactions, reputational damage, and further attacks using the

compromised account as a trusted insider. For accounts with elevated privileges (administrators, support staff, financial approvers), the impact is significantly higher.

E) Recommended Mitigation

Security questions should be deprecated entirely. NIST 800-63b explicitly discourages knowledge-based authentication for account recovery, stating that "the use of knowledge-based authenticators (e.g., security questions) is not recommended for any purpose." The recommended alternative is email-based reset links with time-limited, single-use tokens that expire after a short period (e.g., 15 minutes). For higher assurance applications, require multiple verification factors – for example, send a reset link to email AND a code to SMS or an authenticator app. If security questions must be used (e.g., for legacy system constraints), the questions should have high entropy answers that are not publicly discoverable, and answers should be hashed like passwords. All password reset attempts should be logged, and suspicious patterns are alerted. This aligns with OWASP ASVS requirement 2.2.4.